



# ICS lab测试

Created @December 14, 2025 1:22 PM

一些操作性质：

1. tar -xvf 解压命令

2. gdb:

r或run 开始执行程序

q或quit 退出GDB

ni或next instruction 执行下一条指令，不进入函数

si或step instruction 执行下一条指令，进入函数

b或break 断点

c或continue 继续

j或jump 跳转到程序指定为止

layout asm 显示汇编代码视图

layout regs 显示当前寄存器状态

p或print 打印变量的值

x或examine 打印内存的值

p \$rax # 打印寄存器 rax 的值

p/x \$rsp # 打印栈指针的值，以十六进制显示

p/d \$rsp # 打印栈指针的值，以十进制显示

对于p：x十六进制 d十进制 c字符形式 s字符串形式

x/2x \$rsp

对于x：xdcs检查也是以形式区分，逻辑同上，但是中间可以加数字来确定检查多少个字节

3. Linux基本指令：

下面介绍Linux系统的相关指令：

- 上传文件： `scp [本地文件] ubuntu@[ip]"[文件夹]`，亦可使用VsCode的 `Remote-SSH` 插件拖拽实现。
- 下载文件（到当前目录）： `scp ubuntu@[ip]:/home/...`
- 当前工作目录： `pwd`
- 当前文件夹列表： `ls`
- 进入 `xxx` 文件夹： `cd xxx`
- 返回上级文件夹： `cd ..`
- 创建文件夹： `mkdir [name]`
- 创建文件： `touch [name]`
- 删除： `rm[option][name]`
  - `r` 递归删除一个目录
  - `f` 强制删除
- 压缩与解压缩tar文件： `tar [option][name]`（常用： `tar -xvf [name]`）
  - `x` 解压
  - `c` 选择多个目录/文件进行打包
  - `f [PACKAGE_NAME]` 指定打包后文件名
  - `z` 压缩与解压缩 `tar.gz` 文件
  - `C [TARGET_DIR]` 指导解压目录
- 编译C文件： `gcc [src] -o [dst]`

## Data

1. 使用`~`和`&`实现 `x|y`： `x|y=~((~x)&(~y))`
2. 高n位用1填充：构造一个-1，这是一个32位均为1的二进制数，然后左移即可。使用!n规避移32位的情况
3. 四位二进制数的加法：利用 `x+y=x^y+(x&y)<<1` 来运算，这样进行4次重复相加后就不会出现有进位的项了，所以迭代四次后 `sum=x4|y4`，由于我们使用的是32位int数字，但

是4位加法需要截断，所以最后需要给一个 `mask=0xf`，最后的结果应该为

`res=sum&mask`。

4. 实现一个循环左移，可以指定n位：`rotateLeft(int x, int n)`，分别处理两部分即可，对于高n位部分，`x1=x>>(32-n)`，但是首先要规避移32位，`x1=x<<((33+~n)&32)`，还需要防止最高位为1，导致其高位全是1，需要一个`mask=(1+~1)+(1<n)`，这样的mask就是前面n位是0，后面都是1。对于低位部分，只需左移即可，`x2=x<<n`，最后返回 `x1|x2` 即为结果。
5. 当32位二进制数中1的数量为奇数就返回1：通过折叠抑或技术就可以，抑或相当于不进位的加法，循环做 `x=x+x<<16`，`x=x+x<<8` 直到 `x=x+x<<1`，去最后一位实际上就是所有位的不进位相加值，或者说是相加mod2值，返回最后一位即可。
6. 回文字判断：思路是把x完全反转后和x本身做抑或，如果所有位全为0则返回1.反转则需要一半一半的进行，例如先交换高低16位，再在每一个块中交换高低8位，然后是4位，直到1位。以上过程需要精心设计一下mask，例如 `a=((a>>8)&0x00ff00ff)|((a<<8)&0xff00ff00)`。
7. 计算-x：`~x+1`
8. 如果y比x大1就返回1：`!((y+1+~x)^1)`，然而如果x=Tmin，没有-x与之对应所以这个需要做一下特殊判断
9. 计算3x/4并向0取整：其实主要难点是**向0取整**，`x>>k` 给出x/2^k的下取整，`(x+(1<k)-1)>>k` 给出x/2^k的上取整。所以本题中应该是判断3x为负时就使用 `(x+3)>>2` 计算除法。
10. 如果x<y，就返回1：同号和异号分开考虑，防止Tmax和Tmin出现问题
11. 计算2x，overflow返回对应的Tmin或Tmax：判断sign和移位后的sign是否一样，如果不一样就要返回Tmin或Tmax了，返回那个由sign决定。核心就是x<<1。
12. Mod3：这个咋搞的来着
13. float x计算x/2：使用三个mask将s，exp和frac三部分提取出来，分成四种情况，exp全为1，返回原值即可，exp全为0，则/2后还是exp全为0，这时需要将M/2，记得注意舍入关系。然后对于规整化数，exp最低位为1，直接将exp | frac左移一位，如果exp最低位不是1，则将exp-1即可。
14. 将int x转化为float型：注意向偶数舍入的问题，将出去最大1的x右移8位得到frac部分的值，然后计算round即可
15. 将double转化为int

## 16. 计算 $2^x$

工具：

`dlc`：用于检查你的代码是否符合规范。

```
# 检查是否符合编码规范./dlc bits.c bash
```

`bdd checker`：穷举测试所有可能的输入，完整地检查你的函数是否正确。

```
# 对某个函数进行单元测试./bddcheck/check.pl -f bitXnor# 检查所有函数./bddcheck/check.pl# 检查所有函数，且输出总结信息./bddcheck/check.pl -g bash
```

`driver.pl`：用于评分，检查你的函数是否符合规范且正确。

```
./driver.pl
```

去掉unsigned：用掩码

关于取整：

>> 默认向下取整

向偶：浮点数的逻辑，为了避免偏差

# Bomb&Attack

操作：

./bomb psol.txt 在当前目录下，给bomb这个程序，传递psol.txt这个参数。直到遇到EOF然后转到stdin

objdump -d bomb > bomb.asm 反汇编文件bomb到bomb.asm

objdump: linux的二进制文件分析工具

-d: objdump的选项 -d表示disassemble

>: 输出重定向操作符

objdump -t bomb 显示所有函数的概览（你会看到phase\_1, ..., phase\_6, explode\_bomb, func4, main, etc），看二进制符号表

man objdump 显示objdump的详细用法

man gdb 查看gdb完整手册

man ascii 查看ASCII表

info gas 提供GNU汇编器的完整信息

Ctrl+C终止炸弹运行（这就是SIGINT）

gdbinit里面预先写好就行

炸弹安全化：

linux:

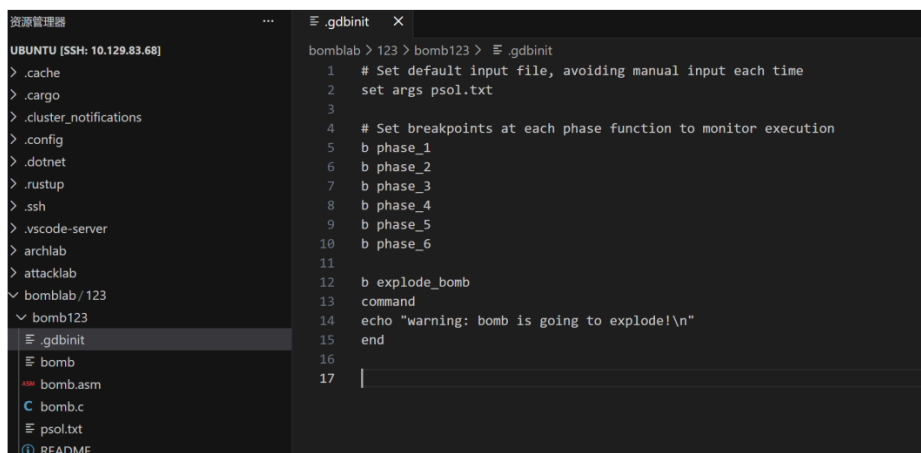
> touch .gdbinit 创建一个.gdbinit文件 >

> mkdir -p ~/.config/gdb 为gdb创建一个专用的配置目录

> echo "set auto-load safe-path /" > ~/.config/gdb/gdbinit 允许gdb预加载文件

```
b phase_1
command
p $rsp # Print the value of the stack pointer
end
```

This script will automatically print the value of the stack pointer when the breakpoint is hit.  
(Maybe you can think of some other useful scripts, awa.)



attack主要的思路就是画栈帧结构，看怎么溢出，具体的有时间再补

哦对，这里面还有一个小端法是贯穿这个lab填地址过程的

# Arch

## 1. 构建三个ys汇编代码实现一些功能

这部分没啥好说的吧，就正常写汇编代码就行，rsum需要使用迭代的方法，所以每次需要将得到的数据扔到栈中，然后继续调用函数本身，记得开头需要.pos 0; pushq stack, %rsp; call main; halt;

bubble需要实现一个冒泡排序，就正常执行就可以了吧

## 2. 使用HCL-rs实现要求的HCL设计

### 2.1 首先在SEQ中增加IOPQ的指令

将IOPQ加入合法指令instr\_valid、需要使用寄存器need\_regids、需要使用常数need\_valC、srcB使用rB、dstE使用rB、aluA使用valC、aluB使用valB、alufun和OPQ放在一起读取ifun、需要set cc。

### 2.2 实现3/4级流水线

示例中实现了2级流水线，取指是一个阶段，其他部分是一个阶段

需要实现3级流水线为取指、译码、其他；4级则是取指、译码、执行和其他。

- 完成pipe\_s3a、pipe\_s3b、pipe\_s3c、pipe\_s3d的三级流水线

只有F D E三个阶段

- a对于数据冒险只是进行停顿/气泡

- f\_pc部分：

D.icode == CALL : D.valC

E.icode == JX && e\_cnd : E.valC （为进行分支预测，所以时JX成立才跳转）

E.icode == RET : e\_valM

默认情况下使用F.valP

- 执行阶段e\_dstE：需要判断cmov是否成立，如果不成立就可以不使用寄存器了，注意流水线寄存器在该步骤的前面，所以e从E中生成

E.icode == CMOVX && !e\_cnd : RNONE;

1 : E.dstE

- 处理数据冒险，a中不使用数据前递。

**首先你应该背下来数据冒险的定义：**bool data\_hazard = d\_srcA != RNONE && d\_srcA in {e\_dstE, e\_dstM} || d\_srcB != RNONE && d\_srcB in {e\_dstE, e\_dstM}; 实际上就是译码阶段得到的数据来源是第三阶段写回的。

D.icode为JX和RET时 f停，如果d没有停，就将d阶段气泡；数据冒险则需要fd停，e气泡

- b使用转发解决数据冒险

- 增加数据前递，则在D阶段的d\_valA和d\_valB中，需要增加前递数据e\_dstE和e\_dstM，判断条件其实就是本阶段目前的d\_scrA == e\_dstE等等
- 三阶段流水线不存在load-use冒险，所以数据前递后不会出现数据冒险问题了。只需要在出现RET和JX时让f停，d气泡即可，不会出现其他情况

- c预测跳转成立，优化控制冒险，主要针对f\_pc

- c进行了跳转预测，PC默认为pred\_pc，所以JX的条件需要改变一下：  
E.icode == JX && !e\_cnd : E.valP
- 数据前递和b中完全一致
- 新增了分支预测错误的判断：**bool branch\_mispred = E.icode == JX && !e\_cnd;**

- 以及ret产生的控制冒险：**bool ret\_harazard = D.icode == RET;** D阶段出现RET则F阶段指令无效，应该放气泡所以在D进行判断
- 如果遇到ret导致的控制冒险就停掉F防止其为D提供错误的指令，然后D上放气泡即可，当然如果同时出现分支错误就可以不处理了，因为分支错误导致其在D上的指令不需要被执行
- 如果遇到分支预测错误，F重新取指即可，E阶段放气泡洗掉D中给E提供的指令。
- d优化寄存器文件读写顺序，使用管线硬件 hardware\_pipe ，合并寄存器读写为 reg\_file ，写后读以避免结构相关
  - 但是实际上在我们需要写的地方完全一样
- 完成pipe\_s4a、pipe\_s4b、pipe\_s4c的四级流水线
  - 分支预测错误仍为 `E.icode == JX && !e_cnd : E.valP` 但是不知道为啥bc变成了 `M.icode == JX && !M.cnd : M.valP`
  - ret的分支在取指中要在M阶段进行了：`M.icode == RET : m_valM`
  - a中只处理了E阶段产生的数据冒险问题 类似于 `d_srcA == e_dstE : e_valE;`  
`d_srcA == m_dstE : m_valE;` bc中给出了完整的考虑M阶段的数据冒险，应该增加：`d_srcA == e_dstE : e_valE;` `d_srcA == m_dstM : m_valM;`
  - ret冒险的判断还要增加，在D和E出现均为ret冒险
  - load-use冒险定义为：`bool load_use_harazard = E.icode in { POPQ , MRMOVQ } && E.dstM in { d_srcA , d_srcB }`
  - 出现ret冒险且不出现分支预测错误时，f停，如果没有出现load-use冒险，则d气泡
  - 分支预测错误仍然是e气泡
  - 出现load-use冒险，则fd停，e气泡
  - 将分支错误的预测推迟到M阶段后上述情况有所变化：如果出现load-use冒险，仍然停fd，e气泡；ret冒险则停f，如果d没有停则气泡d；分支预测错误e气泡，如果d没有停则气泡d

## 2.3 实现程序优化

程序要求是实现一个长度为len的整数数组从src到dst的转移，并返回其中正数的个数



优化空间有两个：ncopy.rs ncopy.js 一个是流水线的优化，一个是程序代码的优化  
ncopy.rs在前面的基础上，改进为5层流水线，增加了W阶段。

在ncopy.js中采用循环展开的方法进行：将拷贝和记数都使用9个元素一组的方法进行，而余数部分采用线性但流水线化的方法，如果不对余数进行处理似乎只有97分

主循环部分：将9个元素放在寄存器上，然后一个一个移动和记录是否为正数

余数部分的要点是，先在rbx中放上要转移的数，然后判断还有多少个剩余，然后再对这个数进行相关的操作，这样可以有效的避免load-use冒险。通过这样的线性处理余数的过程就已经可以到达满分了。

## Cache

writeup:

LRU-policy

- Usage: ./csim-ref [-hv]-s <s>-E <E>-b <b>-t <tracefile>
- h: Optional help flag that prints usage info
- v: Optional verbose flag that displays trace info(详细)
- s <s>: Number of set index bits ( $S = 2^s$  is the number of sets)
- E <E>: Associativity (number of lines per set)
- b <b>: Number of block bits ( $B = 2^b$  is the block size)
- t <tracefile>: Name of the valgrind trace to replay

- The trace files are generated by a Linux program called valgrind.  
E.g. `linux> valgrind --log-fd=1 --tool=lackey -v --trace-mem=yes ls -l`

· `valgrind`: 一个在运行时对程序进行动态分析的工具套件，常用来查找内存错误、内存泄漏或收集执行/内存访问信息。

· `--log-fd=1`: 把 Valgrind 的输出写到文件描述符 1，也就是标准输出 (stdout)。默认 Valgrind 会把日志写到 stderr 或文件，指定这个选项则改为写到 stdout。

· `--tool=lackey`: 指定使用 Valgrind 的 lackey 工具 (lackey 是一个简单的跟踪/分析工具，用来记录指令执行和内存访问等)。

· `-v` (或 `--verbose`): 以详细模式运行，输出更多信息 (视工具而定)。

· `--trace-mem=yes`: 告诉 lackey 工具记录内存访问 (load/store) 轨迹。

· `ls -l`: 这是要被 Valgrind 监视并收集内存访问的目标可执行程序 (在这里举例为 `ls -l`，即列出目录详细信息的命令)

(s = 5, E = 1, b = 5) 32132 直接映射高速缓存,可存放328个int

- (1)  $32 * 32$

先复制再转置 (B中的88块都在缓存中全部命中!)

- (2)  $64 * 64$

4\*4处理，分对角线与非对角线

- (3)  $60 * 68$

56\* 64 8\* 8处理，余下边再4\*4处理

## Shell

### 1.I/O重定向

例如，输入命令行 `tsh> /bin/ls > foo`

`> foo` 是 shell 的重定向符号，表示将命令的标准输出（stdout）写入文件 `foo`（如果 `foo` 存在则覆盖，不存在则创建）。

所以这条命令的意思是：**运行 `ls`，把原本输出到屏幕的内容保存到文件 `foo` 里，屏幕上不会显示 `ls` 的结果。**

类似地，`tsh> /bin/cat < foo`

将文件 `foo` 的内容显示在标准输出上。

`< foo` 是输入重定向符号，表示将文件 `foo` 的内容作为命令的标准输入（stdin）。

所以这条命令的意思是：**运行 `cat`，并从文件 `foo` 读取内容，`cat` 将这些内容输出到屏幕上，等价于 `cat foo`（不过这里的 `cat` 没有参数，是通过重定向给它输入）**

## Writeup:

一些要求：

4. 输入 `ctrl-c`（`ctrl-z`）应使你的 shell 向当前前台作业及其任何后代（例如，它 fork 的任何子进程）发送 `SIGINT`（`SIGTSTP`）信号。如果没有前台作业，则该信号应无效。

5. 如果命令行以 `&` 结尾，则 `tsh` 应在后台运行该作业。否则，它应在前台运行该作业。

6. 每个作业可以通过进程 ID（PID）或作业 ID（JID）来标识，JID 是由 `tsh` 分配的正整数。JID 应在命令行中以前缀 `"%"` 表示。

例如，`"%5"` 表示 JID 5，`"5"` 表示 PID 5。

tsh 应支持以下内置命令：

1. quit 命令终止 shell。
2. jobs 命令列出所有后台作业。
3. bg job 命令通过向其发送 SIGCONT 信号来重启 job，然后在后台运行它。job 参数可以是 PID 或 JID。
4. fg job 命令通过向其发送 SIGCONT 信号来重启 job，然后在前台运行它。job 参数可以是 PID 或 JID。
5. kill job 命令通过向每个相关进程发送 SIGTERM 信号来杀死作业列表中的 job 或进程组。job 参数可以是 PID 或 JID。

如果 job 参数是正数，你的 shell 应杀死相应的作业。如果 job 不存在，你的 shell 应打印 "%JID: No such job" 或 "(PID): No such process"。

请注意，如果你得到一个负的 job 参数，例如 kill %-1 或 kill -15213，你的 shell 应杀死 JID 为 %-JID 或 PID 为 -PID 的作业的进程组。

但是由于前面说的无进程组，所以可以直接对job参数取绝对值

注意作业列表上的竞争条件。

在 fork 之后，你不能对父进程和子进程的执行顺序做出任何假设。特

别是，你不能假设当父进程从 fork 返回时子进程仍在运行。

此外，信号处理程序与程序并发运行，并且可以在任何地方中断程序，除非明确屏蔽信号的接收。

禁止在等待信号时在紧密循环中空转（例如，"while (1);"）

这样做极其浪费处理器时间。在紧密循环中调用 sleep 也不合适。

相反，应该在任何紧密循环中使用 sigsuspend 函数。详见教科书。

当shell的子进程死亡时，必须在有限时间内回收它们。这意味着你不能等到前台进程结束或用户输入内容后才回收。

不应该在多个地方调用 waitpid。这会导致大量潜在的竞争条件，并使 shell 变得不必要地复杂。

几个点：

在 eval 中，父进程必须在 fork 子进程之前使用 sigprocmask 阻塞 **SIGCHLD**、**SIGINT** 和 **SIGTSTP** 信号，然后通过调用 addjob 将子进程添加到作业列表后，再次使用 **sigprocmask** 解除阻塞这些信号。

由于子进程继承父进程的阻塞向量mask，子进程必须在 exec 新程序之前解除阻塞这些信号。

子进程还应恢复 shell 忽略的信号默认处理程序。父进程需要以这种方式阻塞信号，以避免竞争条件（例如，子进程在父进程调用 `addjob` 之前被 `sigchild_handler` 回收（从而从作业列表中移除））

当你在标准 Linux shell 中运行你的 shell 时，你的 shell 在前台进程组中运行。如果你的 shell 然后创建一个子进程，默认情况下该子进程也将是前台进程组的成员。由于输入 `ctrl-c` 会向前台组中的每个进程发送 `SIGINT`，输入 `ctrl-c` 将向你的 shell 以及你的 shell 创建的每个进程发送 `SIGINT`，这显然是不正确的。

解决方法如下：在 `fork` 之后，但在 `execve` 之前，子进程应调用 `setpgid(0, 0)`，这将子进程放入一个新的进程组，其组 ID 与子进程的 PID 相同。这确保了前台进程组中只有一个进程，即你的 shell。

当你输入 `ctrl-c` 时，shell 应捕获产生的 `SIGINT`，然后将其转发到适当的前台作业（更准确地说，是包含前台作业的进程组，在 lab 条件约束下，这个进程组里面只有一个进程）

## 实现一些细节

Eval:

eval函数的大致流程：

1. 解析命令行 (使用已经写好的 `parseline`)
2. 处理输入重定向 (如果有)
3. 处理输出重定向 (如果有)
4. 判断命令类型 (内置命令 or 外部命令)
5. 执行相应命令 (7种可能)
6. 恢复输入输出重定向 (如果有)

`dup`保存就文件no后，先输入输出重定向，然后一堆switch

nohup:

```
sigset_t mask, prev;  
Sigemptyset(&mask);  
Sigaddset(&mask, SIGHUP);  
  
// 阻塞 SIGHUP  
Sigprocmask(SIG_BLOCK, &mask, &prev);  
  
// 执行命令，子进程会继承阻塞状态  
eval(cmdline + 6);  
  
// 恢复原来的信号掩码  
Sigprocmask(SIG_SETMASK, &prev, NULL);
```

这里的 6 是 "nohup " 字符串的长度（包括空格）

比如，输入的cmdline是  
"nohup sleep 100 &\n"

那么cmdline + 6 指向：  
"sleep 100 &\n"

剩下几个handler：

sigchld

sigchld\_handler-每当子作业终止（变成僵尸进程），或者因为收到SIGSTOP、SIGTSTP、SIGTTIN或SIGTTOU信号而停止时，内核都会向shell发送一个sigchld。处理程序会获取所有可用的僵尸子节点，但不会等待任何其他当前正在运行的子节点终止。

```
// 使用WNOHANG和WUNTRACED选项非阻塞地回收子进程
while((pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0) {
    Sigprocmask(SIG_SETMASK, &mask_all, &prev_all);
```

……经过一系列对子进程的回收处理

```
Sigprocmask(SIG_SETMASK, &prev_all, NULL);
```

操作：

1. 去掉vscode报错：将其中cStandard 改为 gnu11

知识：

阅读 `tsh.c` 文件源码，找到 `job` 相关结构体：

```
1 struct job_t
2 {
3     pid_t pid;          /* The job struct */
4     int jid;            /* job PID */
5     int state; /* UNDEF, BG, FG, or ST */ // 即未定义, 后台, 前台, 挂起
6     char cmdline[MAXLINE]; /* job ID [1, 2, ...] */
7 };
8 struct job_t job_list[MAXJOBS]; /* command line */
```

Copy

```
1 struct cmdline_tokens
2 {
3     int argc;           /* Number of arguments */
4     char *argv[MAXARGS]; /* The arguments list */
5     char *infile; /* The input file */ // 输入重定向
6     char *outfile; /* The output file */ // 输出重定向
7     enum builtins_t
8     {
9         /* Indicates if argv[0] is a builtin command */
10        BUILTIN_NONE, // 啥都没有
11        BUILTIN_QUIT, // 退出
12        BUILTIN_JOBS, // 展示全部job
13        BUILTIN_BG,   // 转为后台
14        BUILTIN_FG,   // 转为前台
15        BUILTIN_KILL, // 杀死job
16        BUILTIN_NOHUP
17    } builtins; // 忽视SIGUP, 启动新进程
18 };
```

Copy

这意味着，命令行参数被组织为一个 `cmdline_tokens` 类，其中 `argc` 表示 **参数个数**，`argv` 表示 **参数列表**（由课本知识，`argv[0]` 表示命令名称），`infile` 跟 `outfile` 是读写重定向文件名（见后文），`builtins` 是内建命令。



| 信号                   | 信号全称                       | 默认行为    | 相关事件             |
|----------------------|----------------------------|---------|------------------|
| <code>SIGINT</code>  | Interrupt Signal           | 终止      | 来自键盘的中断 (Ctrl-C) |
| <code>SIGILL</code>  | Illegal Instruction Signal | 终止并转储内存 | 非法指令             |
| <code>SIGFPE</code>  | Floating Point Exception   | 终止并转储内存 | 浮点异常             |
| <code>SIGKILL</code> | Kill Signal                | 终止      | 杀死进程             |
| <code>SIGSEGV</code> | Segmentation Fault Signal  | 终止并转储内存 | 无效的内存引用 (段故障)    |
| <code>SIGUSR1</code> | User-defined Signal 1      | 终止      | 用户定义的信号 1        |
| <code>SIGUSR2</code> | User-defined Signal 2      | 终止      | 用户定义的信号 2        |

| 信号                   | 信号全称                        | 默认行为           | 相关事件                           |
|----------------------|-----------------------------|----------------|--------------------------------|
| <code>SIGALRM</code> | Alarm Clock Signal          | 终止             | 来自 <code>alarm</code> 函数的定时器信号 |
| <code>SIGCHLD</code> | Child Status Changed Signal | 忽略             | 一个子进程停止或终止                     |
| <code>SIGCONT</code> | Continue Signal             | 继续执行           | 继续进程, 如果该进程停止                  |
| <code>SIGSTOP</code> | Stop Signal                 | 停止直到下一个SIGCONT | 不是来自终端的停止信号                    |
| <code>SIGTSTP</code> | Terminal Stop Signal        | 停止直到下一个SIGCONT | 来自终端的停止信号 (Ctrl-Z)             |

## 封装函数

在开始编写eval函数之前, 回顾课本, 为了便于Debug以及输出错误信息, 需要将给定函数封装为大写开头函数 (便于区分)。

# Malloc

